

크롤러 유지보수 트러블슈팅

커뮤니티 수집 크롤러의 개편 내성(resilience) 개선 사례 · 김재연

* 본 문서의 코드·사례는 실제 사내 크롤러 엔진 소스가 아니라, 업무 중 적용한 문제 해결 방식과 설계 아이디어를 범용 코드로 재구성한 예시입니다. 크롤링 대상 사이트명·발주처·내부 시스템 정보는 모두 익명화했습니다.

개요

수집 개 커뮤니티 사이트를 수집하는 크롤러를 운영·유지보수하면서, 사이트 개편과 봇 차단으로 반복되던 수집 장애를 개별 수정이 아닌 구조 개선으로 줄인 작업입니다. 핵심은 하나의 셀렉터에 의존하지 않고, 값마다 신뢰도 순으로 여러 추출 전략을 세워 실패 시 자동 폴백하도록 설계한 것입니다.

추출 전략 우선순위 (다단계 폴백)

순위	전략	이유
1	JSON-LD (application/ld+json)	구조화 데이터 — 화면 클래스가 바뀌어도 유지
2	OpenGraph / meta 태그	대부분 사이트가 공통 보유
3	안정 시맨틱 셀렉터 (data-*, id 접미사)	빌드에 의존하지 않는 요소
4	정규식 기반 텍스트 추출	최후의 수단

→ 빌드마다 바뀌는 해시 클래스(예: `svelte-a1b2c3`)는 셀렉터에 절대 쓰지 않는다 — 이 원칙으로 SPA 개편 시 재발하던 장애를 크게 줄였습니다.

사례 1. SPA 개편으로 무너진 셀렉터 복구

문제 (Problem)

한 커뮤니티가 서버렌더링 방식에서 SvelteKit/shadcn 기반 SPA로 전면 개편되면서 기존 클래스 기반 셀렉터가 전부 깨졌다. 특히 빌드마다 `svelte-XXXXXX` 형태의 해시 클래스가 새로 생성돼, 클래스를 다시 맞춰도 다음 배포에서 또 깨지는 구조였다.

분석 (Analysis)

- 해시 클래스는 근본적으로 신뢰할 수 없는 셀렉터임을 확인.
- 반면 JSON-LD, og: 메타태그, data-slot=id 접미사 등 빌드에 의존하지 않는 요소가 남아 있음을 발견.
- 크롤러 요청이 영문 로케일로 렌더되어 날짜가 AM/PM으로 오는 부수 문제도 확인.

해결 (Solution)

- 값마다 다단계 폴백 체인 설계: JSON-LD → 메타태그 → 안정 셀렉터 → 정규식.
- 해시 클래스는 셀렉터에서 완전히 배제.
- 게시판별 접두어가 붙는 본문 id를 접미사 매칭(`id$=post-content`)으로 흡수.
- 날짜 파서를 오전/오후·AM/PM·ISO 8601 모두 대응하도록 통일.

결과 (Result)

개편 후에도 수집이 유지되고, 이후 재배포로 클래스가 또 바뀌어도 셀렉터가 깨지지 않는 구조를 확보. 개편 때마다 반복하던 수동 대응 부담을 구조적으로 줄였다.

핵심 코드 (개념 재구성)

```
# 값마다 신뢰도 순으로 폴백 — 앞이 실패하면 다음 전략으로
title, used = first_nonempty(
    lambda: ld.get('title'),           # 1) JSON-LD
    lambda: og_content('og:title'),   # 2) 메타태그
    lambda: sel_text('[data-slot=card-title]'), # 3) 안정 셀렉터
    lambda: sel_text('h1'),           # 4) 최후 수단
)
# 본문: 게시판별 접두어(economy-/free-)에 관계없이 접미사로 매칭
sel('[id$=post-content]')
```

사례 2. 구조화 데이터(JSON-LD) 우선 추출

문제

여러 사이트에서 화면 마크업이 조금만 바뀌어도 본문·날짜 추출이 실패했다. HTML 태그 위치에만 의존하는 방식의 한계였다.

분석

상당수 사이트가 SEO 목적으로 `application/ld+json`에 제목·본문·작성일·댓글수를 이미 담고 있었고, 이 데이터는 화면 디자인이 바뀌어도 잘 유지된다는 점에 주목했다.

해결

- JSON-LD를 1순위 소스로 채택. 배열/단일 객체를 모두 처리하고 `@type`이 `Article·DiscussionForumPosting` 등인 노드를 선별.

- 깨진 JSON 은 조용히 건너뛰고 다음 후보로 폴백.
- 댓글수는 comment[] 배열이 잘려 담기는 경우가 있어, 집계 필드 interactionStatistic(CommentAction)만 신뢰.

결과

마크업 변경에 대한 내성이 크게 향상. 이후 유사 사이트 추가 시에도 같은 패턴을 재사용해 대응 시간을 단축했다.

사례 3. 게시판 추가에 따른 URL 패턴 확장

문제 · 분석

수집 대상 게시판을 추가하거나 사이트가 URL 체계를 바꿀 때마다 목록·게시글 판별 정규식을 수정해야 했다. 케이스가 반복적이었다 — 게시판 편입(OR 분기 추가), 쿼리스트링 → path 개편, 게시글 꼬리 파라미터로 인한 중복 수집.

해결

- 목록/게시글 판별 정규식을 사이트별 레지스트리로 모아 관리 — 게시판 추가 = 규칙 한 줄 추가.
- 게시글 URL 은 쿼리스트링 절단·상대경로 정규화로 중복 수집 제거.
- 느슨하던 게시글 정규식을 명시적으로 좁혀 오탐 감소.

코드 (개념 재구성)

```
# 게시판 하나 추가 = OR 분기 한 줄
# .../pt?page=N      -> .../(pt|name)?page=N
# id=fun&page=N      -> id=(fun|news2)&page=N
# 게시글 URL 정규화: 꼬리 파라미터 제거로 중복 제거
def clean_article_url(url):
    return url.replace('..', '').split('?')[0]
```

결과

게시판 추가·사이트 개편 대응이 규칙 등록만으로 가능해져 유지보수 비용이 감소. 중복 수집으로 인한 데이터 품질 저하도 완화했다.

사례 4. 지저분한 실데이터 정규화

문제 · 분석

사이트마다 날짜·조회수 표기가 제각각이었다. 날짜는 2026.04.26 (일) 15:54 / 2026-03-09T15:17:00+09:00 / 2026 년 3 월 9 일 오후 3:17 등이 혼재했고, 조회수는 콤마·라벨이 섞이거나 값이 없을 때 빈 문자열·NaN 이 발생했다.

해결 · 결과

- 날짜를 YYYY-MM-DD HH:MM:SS 단일 포맷으로 통일하는 파서 작성(오전/오후·AM/PM·타임존 처리 포함).
- 숫자는 비슷자 문자를 제거하고 빈 값/NaN 은 0 으로 방어.

수집 데이터의 포맷이 일관돼 후속 분석·적재의 안정성이 향상됐다.

사례 5. 외부 수집기 장애 시 대체 소스 전환 (failover)

문제 · 분석

특정 지표(예: 외부 채널의 구독자 수)를 외부 수집기에 의존해 가져오던 중 해당 수집기에 장애가 발생해 수집이 중단됐다. 단일 외부 소스에 의존하는 구조라, 그 소스가 죽으면 지표 자체가 비는 것이 문제였다. 동일 지표를 얻을 수 있는 대체 경로가 있음을 확인했다.

해결 · 결과

- 외부 수집 사이트를 대체 소스로 전환하고, 해당 지표 수집용 목록 화면 템플릿을 그에 맞게 수정.
- 전환 후 정상 수집 여부를 모니터링으로 확인 — 장애 상황에서도 지표 수집을 재개.

외부에 의존하는 수집 지표는 대체 경로를 염두에 둔 설계가 필요하다는 점을 실제 대응으로 확인했다. 장애는 내 코드가 아니라 외부에서 온다.

이 작업이 보여주는 역량

- 운영 중 반복 장애를 개별 수정이 아닌 구조 개선으로 해결
- 외부 시스템(사이트) 변화에 견디는 방어적 설계
- 지저분한 실데이터를 일관 포맷으로 정규화하는 처리력
- 단일 외부 소스 의존을 대체 경로 전환(failover)으로 완화하는 운영 감각
- 유지보수 비용을 낮추는 설정 중심(레지스트리) 구조화

전체 예시 코드(Python·JS)와 README 는 함께 제공되는 GitHub 저장소 형태의 파일에 포함되어 있습니다.